Consultorio Veterinario

Proyecto MVC con PHP y MySQL · 17 funciones para resolver

De qué se trata

Te damos una aplicación web **completa y funcionando**: la base de datos, la conexión, el ruteo, los controladores, las vistas y el diseño. Todo eso ya está resuelto y no hay que tocarlo.

Lo único que falta son **las consultas a la base de datos**. Los modelos tienen los nombres de las funciones y el contrato de cada una escrito, pero el cuerpo está vacío. Tu trabajo es escribir esas 17 consultas. A medida que las resolvés, la aplicación se va encendiendo sola.

Puesta en marcha — 10 minutos

1. Copiá la carpeta `veterinaria-mvc` dentro de `C:\xampp\htdocs`
2. Abrí el panel de XAMPP y arrancá **Apache** y **MySQL**.
3. Entrá a `http://localhost/phpmyadmin` y, en la pestaña **Importar**, ejecutá los tres archivos **en este orden**: `sql/01_ddl.sql` → `sql/02_dcl.sql` → `sql/03_datos.sql`
4. Entrá a `http://localhost/veterinaria-mvc/`

Vas a ver la aplicación andando, pero con carteles amarillos por todos lados que dicen qué función falta resolver. Ese es el punto de partida.

Sobre los tres archivos .sql

01_ddl.sql — DDL, *Data Definition Language*. Crea la base y las cinco tablas con sus claves foráneas. Es el "molde".

02_dcl.sql — DCL, *Data Control Language*. Crea el usuario `vet_app` y le da permisos de `SELECT`, `INSERT`, `UPDATE` y `DELETE`, pero no de `CREATE`, `DROP` ni `ALTER`. La aplicación se conecta con ese usuario, nunca con root.

03_datos.sql — DML de carga. Mete 6 especies, 5 veterinarios, 10 clientes, 18 mascotas y 23 consultas. Leelo: es tu referencia para saber qué tiene que devolver cada función.

El modelo de datos



- Una **especie** tiene muchas **mascotas** (perro, gato, conejo...).
- Un **cliente** tiene muchas **mascotas** (es el dueño).
- Una **mascota** tiene muchas **consultas** (su historial clínico).
- Un **veterinario** atiende muchas **consultas**.

Las columnas exactas de cada tabla están en `sql/01_ddl.sql` y también repetidas al principio de cada modelo, para que no tengas que ir y venir.

Qué tocás y qué no

```
veterinaria-mvc/
├── sql/ ···························· resuelto · leelo
├── public/ ·························· resuelto · no tocar
└── app/
    ├── config/config.php ··········· resuelto · no tocar
    ├── nucleo/ ····················· resuelto · LEER Modelo.php
    ├── controladores/ ··············· resuelto · no tocar
    ├── vistas/ ····················· resuelto · no tocar
    └── modelos/
        ├── Veterinario.php ····· EJEMPLO RESUELTO · tu referencia
        ├── Especie.php ·········· 2 ejercicios
        ├── Cliente.php ·········· 5 ejercicios
        ├── Mascota.php ·········· 5 ejercicios
        └── Consulta.php ········· 5 ejercicios
```

Solo se editan los cuatro archivos marcados en verde. Si te dan ganas de cambiar una vista porque "así queda mejor", anotalo y comentámelo, pero no lo toques: la corrección asume que las vistas están como te las di.

Antes de escribir la primera línea: leé Modelo.php

En `app/nucleo/Modelo.php` está explicado el método `$this->consulta()`, que es el que vas a usar en las 17 funciones, con ejemplos de `fetch()`, `fetchAll()` y parámetros preparados. Y en `app/modelos/Veterinario.php` tenés cuatro funciones ya resueltas del mismo estilo que las que te tocan.

Los 17 ejercicios

Nivel 1 — Consultas de una sola tabla BÁSICO

#	FUNCIÓN	QUÉ HACE	SQL QUE USA
1	`Especie::todas()`	Lista las especies por nombre	SELECT + ORDER BY
2	`Especie::conCantidadMascotas()`	Cada especie con cuántas mascotas tiene, incluidas las que tienen cero	LEFT JOIN + COUNT + GROUP BY
3	`Cliente::todos()`	Lista los clientes por nombre	SELECT + ORDER BY
4	`Cliente::buscarPorId()`	Un cliente, o null si no existe	SELECT + WHERE + parámetro
5	`Cliente::buscar()`	Busca por nombre, documento o email, parcial	LIKE con comodín
6	`Cliente::contar()`	Cuántos clientes hay (devuelve un número)	COUNT(*)
7	`Cliente::crear()`	Da de alta un cliente y devuelve su id	INSERT

Nivel 2 — Uniendo tablas INTERMEDIO

#	FUNCIÓN	QUÉ HACE	SQL QUE USA
8	`Mascota::todasConDuenoYEspecie()`	Todas las mascotas con nombre de dueño y de especie	JOIN de 3 tablas
9	`Mascota::porCliente()`	Las mascotas de un cliente	JOIN + WHERE
10	`Mascota::verFicha()`	Ficha completa de una mascota	JOIN de 3 + WHERE
11	`Mascota::pesoPromedioPorEspecie()`	Cuántas hay y cuánto pesan en promedio, por especie	JOIN + AVG + GROUP BY
12	`Mascota::actualizarPeso()`	Cambia el peso y avisa si cambió algo	UPDATE + rowCount()

Nivel 3 — Agregados y fechas AVANZADO

#	FUNCIÓN	QUÉ HACE	SQL QUE USA
13	`Consulta::ultimas()`	Las últimas consultas atendidas	JOIN de 4 + LIMIT
14	`Consulta::historialDeMascota()`	Historial clínico de una mascota	JOIN + WHERE + ORDER
15	`Consulta::facturacionPorVeterinario()`	Cuánto facturó cada veterinario	SUM + COUNT + GROUP BY
16	`Consulta::mascotasSinVisitaReciente()`	Las que hace mucho no vienen, y las que nunca vinieron	LEFT JOIN + MAX + HAVING + DATEDIFF
17	`Consulta::registrar()`	Registra una consulta nueva	INSERT

El contrato completo de cada función —qué recibe, qué devuelve y con qué nombres de campo exactos— está en el comentario que hay arriba de cada una, en el archivo del modelo. No los repito acá: leelos ahí, que es donde vas a estar trabajando.

Cómo trabajar cada ejercicio

1. **Leé el contrato.** Fijate qué recibe, qué tiene que devolver, y sobre todo **con qué nombres de campo**. Si la vista espera `'dueno'` y vos devolvés `'nombre_cliente'`, no va a mostrar nada.
2. **Escribí la consulta en phpMyAdmin primero.** Pestaña SQL, la escribís, la corrés y mirás el resultado. Hasta que ahí no dé bien, no la pases al código.
3. **Pasala a PHP.** Guardala en una variable `$sql` y devolvela con `$this->consulta($sql)->fetchAll()` o `->fetch()` según corresponda.
4. **Borrá la línea del `return` provisorio** que dejamos abajo del TODO.
5. **Recargá la página** del navegador y mirá si el cartel amarillo desapareció.

La barra negra del pie es tu mejor herramienta

En cada página, abajo de todo, hay una barra que muestra el recorrido `URL → Controlador → Modelos → Vista` y, desplegable, **todas las consultas SQL que se ejecutaron** con sus parámetros.

Si una página no muestra datos, mirá ahí: si no aparece ninguna consulta, es que tu función todavía devuelve el array vacío. Si aparece pero trae 0 filas, el problema está en el SQL.

Errores frecuentes

SÍNTOMA	QUÉ REVISAR
La página sigue mostrando el cartel amarillo	¿Borraste la línea `return [];` que estaba abajo del TODO?
"No se pudo conectar a la base de datos"	MySQL apagado, o no corriste `02_dcl.sql` y el usuario `vet_app` no existe.
La tabla sale vacía pero la consulta anda en phpMyAdmin	Los nombres de los campos. La vista busca claves exactas: `'dueno'`, `'especie'`, `'cantidad_mascotas'`...
Aparecen menos filas de las esperadas	Probablemente usaste INNER JOIN donde iba LEFT JOIN.
Error "not in GROUP BY"	Toda columna del SELECT que no sea COUNT, SUM o AVG tiene que estar listada en el GROUP BY.
Página en blanco	Error de sintaxis en PHP: falta un `;` o una comilla sin cerrar.

Cómo se verifica que está bien

Con las 17 resueltas, la aplicación tiene que mostrar:

- En **Especies**: las 6 especies en las dos tablas. En la de la derecha, Tortuga y Loro aparecen con 1 mascota cada una y ninguna especie queda afuera.
- En **Clientes**: 10 clientes. Buscar `lu` devuelve a Lucía Fernández.
- En **Mascotas**: 18 mascotas, cada una con su dueño y su especie.
- En la ficha de **Rocco**: 3 consultas en el historial.
- En **Consultas**: la facturación de 4 veterinarios (la Dra. Ferrán está inactiva y nunca atendió, así que no aparece).
- En **Sin control** con 120 días: 9 mascotas, y arriba de todo Copito y Pipo, que nunca tuvieron una consulta.

Entrega

Una carpeta comprimida con el proyecto completo. Se corrige:

- Que las 17 funciones devuelvan lo que pide el contrato.
- Que **todos** los datos que vienen de afuera vayan como parámetro preparado. Una sola concatenación dentro del SQL invalida el ejercicio.
- Que las consultas estén indentadas y se puedan leer.
- Que no hayas tocado los archivos marcados como resueltos.

Desafíos opcionales

A. Agregá a `Mascota` una función `edadEnAnios($id)` que calcule la edad a partir de `fecha_nacimiento`, y mostrala en la ficha.

B. En `Consulta`, agregá `motivosMasFrecuentes()` que devuelva los motivos ordenados por cantidad de veces que aparecen.

C. Agregá un filtro por especie al listado de mascotas. Vas a tener que tocar el controlador y la vista: anotá qué cambiaste y por qué.